Databricks Certified Data Engineer Associate

Comprehensive Revision Notes

Author: Aditya Bhamre

Section 1: Databricks Architecture & Compute

- **Cluster Fundamentals:** A cluster is a group of nodes or computers working together like a single entity. It consists of a Master (Driver) node and multiple worker nodes.
- **Databricks Runtime Version (DBR):** A virtual machine image that comes with preinstalled libraries. It contains specific versions of Spark, Scala, and other essential data engineering libraries.
- **Photon Acceleration:** A vectorized query engine developed in C++ to drastically enhance Spark performance for SQL workloads.
- **Autoscaling:** Enabling 'Autoscaling' allows you to set a minimum and maximum range of worker nodes to optimize compute costs based on workload.
- **Databricks Units (DBU):** A unit of processing capability per hour. The total cost is calculated by multiplying the configuration rate by the DBU/h consumed by the VM.
- **Logging:**
- **Event Log:** Captures all events that have happened with a cluster (e.g., created, terminated, edited, running).
- **Driver Logs:** Logs generated within the cluster notebooks and libraries.
- **Notebook Features:**
- Supports Python (default), SQL, Scala, and R.

- **Magic Commands:** Built-in commands identified by `%` at the start of a cell that provide the same output regardless of the notebook's default language.
- `%md` : Markdown magic command for formatted text (bold, italicized, ordered/unordered lists, images using links, tables, embedded HTML).
- `%run` : Used to run another notebook in the current notebook. Highly useful for building modular code.
- `%fs` : Systems command for file operations (e.g., `%fs ls` for listing files).
- **Databricks Utilities (`dbutils`):** Used for configuring and interacting with the environment. It is more useful than `%fs` commands because `dbutils` can be seamlessly embedded as part of Python code.
- `dbutils.help()` : Displays commands for credentials, data, jobs, library, meta, notebook, preview, secrets, and widgets.
- `dbutils.fs.ls("path")` : Lists files in a directory.

Section 2: Delta Lake Core Concepts

- **Overview:** Delta Lake is an open-source storage framework that brings reliability to data lakes, enabling the Lakehouse architecture (unifying data warehouse and advanced analytic capabilities).
- **Anatomy of a Delta Table:** Consists of underlying storage files (one or more data files in Parquet format) + a Transaction Log. Delta Lake is the default format on Databricks; there is no need to explicitly use the `USING DELTA` keyword.
- **Delta Transaction Log (`_delta_log`):**
- An ordered record of every transaction performed on the table since its creation.
- Serves as the single source of truth.
- Every time you query a table, Spark checks the transaction log to retrieve the most recent version of the data.
- Each committed transaction is stored in a JSON file containing: operations performed (insert), predicates used (conditions, where, join), and data files affected (added/removed).
- **Key Advantages:**

- Brings ACID transactions to object storage.
- Handles scalable metadata.
- Provides a full audit trail of all changes.
- Builds upon standard open data formats (JSON/Parquet).
- **Immutability:** Parquet files themselves are immutable; once created, they cannot be modified.

Section 3: Time Travel, Optimization, & Data Layout

Time Travel

Operations on a Delta table are automatically versioned. You can query older versions of data using the `DESCRIBE HISTORY <table_name>` command.

- **By Timestamp:** `SELECT * FROM table TIMESTAMP AS OF "2019-01-01"`
- **By Version:** `SELECT * FROM table VERSION AS OF 36` OR `SELECT * FROM table@v36`
- **Rollback:** `RESTORE TABLE <table_name> TO VERSION AS OF 36`

Performance Optimization

- **`OPTIMIZE <table_name>` :** Compacts small files into larger, optimized sizes.
- **`ZORDER BY <column-name>` :** Colocating and reorganizing column information in the same set of files. Z-order indexing is used by data-skipping algorithms to extremely reduce the amount of data that needs to be read.
- **`VACUUM <table_name> [RETAIN <retention_period>]` :** Cleans up unused data files and uncommitted files not in the latest table state.
- Default retention period is **7 days**.
- Running vacuum prevents Time Travel for versions older than the retention period.

Data File Layout

- Optimizing the layout helps in leveraging data-skipping algorithms.
- **Liquid Clustering:** An improved version of Z-order indexing with more flexibility and better performance. Dynamically groups data and adapts to prevent the small files problem.
- *Note:* Clustering is not compatible with partitioning or Z-order.

- `ALTER TABLE <table> CLUSTER BY (<columns>)`

Section 4: Unity Catalog & Governance

- **Metastore:** A repository of metadata (databases, tables). Unity Catalog is the default central metastore.
- **Catalog Hierarchy:** `Catalog > Schema (Database) > Table/View`
- `USE CATALOG <catalog_name>` : Command for operations executed under this catalog.
- `CREATE SCHEMA` is equivalent to `CREATE DATABASE` .
- **Table Types:**
- **Managed Table (Default):** Databricks manages the metadata and data. Dropping it deletes both.
- **External Table:** Databricks manages metadata, but data is kept in an external location. Dropping it leaves data files intact.
- **Cloning:** Useful for test environments where data modification will not affect the source table. Both are separate from the source.
- **Deep Clone:** Copies data + metadata.
- **Shallow Clone:** Copies only metadata (Delta logs).
- **Table Constraints:** Enforce data quality.
- `ALTER TABLE <table-name> ADD CONSTRAINT <name> CHECK (date > '2010-01-01')`
- **Governance Controls:**
- `GRANT / REVOKE` statements manage data access controls.
- Unity Catalog ABAC (Attribute-Based Access Control) applies row and column masks for every column tagged with confidential info.
- `dbutils.secrets.get()` is used to manage and retrieve external database passwords securely.

Section 5: Views Comparison

A view is a virtual table that has no physical data; it is a saved SQL query that is executed each time a new query is run.

Feature	Classic (Stored) View	Temporary View	Global Temp View
Persistence	Persisted in Metastore	Not persisted	Not persisted
Scope	Cluster scoped	Tied to Spark Session	Cluster scoped
Database Location	Standard catalog schemas	N/A	Added to `global_temp` database
Dropped When...	Dropped explicitly via `DROP VIEW`	Dropped when session ends	Dropped when cluster is restarted

---

Section 6: Medallion Architecture & Lakeflow Pipelines

Medallion Architecture

- **Bronze:** Raw data (Ingested filtered batch/streaming).
- **Silver:** Filtered / cleansed / validated data (Augmented datasets).
- **Gold:** Enriched data (Aggregated for ML/BI).

Databricks Lakeflow

A unified solution that simplifies all aspects of data engineering (data ingestion + transformation + orchestration). It addresses challenges of building large-scale ETL pipelines.

- **Lakeflow Connect:** Ingestion engine. Low code/no code connectors.
- *Managed Connectors (SaaS):* Salesforce, ServiceNow, Workday, Google Analytics.
- *Database Connectors (CDC):* PostgreSQL, SQL Server, Oracle (syncs row-level changes: inserts, updates, deletes).

- *Standard Connectors:* Requires writing custom Python or SQL code.
- *Partner Connect:* 3rd party services (Fivetran, dbt, Informatica).
- **Lakeflow Pipelines:** Declarative ETL development.
- **Lakeflow Jobs:** Workflow orchestration.

Section 7: Data Ingestion (Auto Loader & COPY INTO)

Ingestion Modes

1. **Batch:** Load data in batches of rows based on schedules. Reprocesses all records with each run. (Techniques: `CTAS`, `spark.read.load()`).
2. **Incremental Batch:** Only new data is ingested; previously loaded records are skipped for faster ingestion. (Techniques: `COPY INTO`, Structured Streaming in triggered mode).
3. **Streaming:** Continuously load micro-batches of data at very short, frequent intervals (Kafka, Amazon Kinesis). (Techniques: Structured Streaming in continuous mode).

Auto Loader vs. COPY INTO

Feature	COPY INTO	Auto Loader (`cloudFiles`)
Method	SQL command	Spark Structured Streaming
Function	Idempotently & incrementally loads new files	Incrementally ingests files
Scale Target	Thousands of files	Millions/Billions of files
Efficiency	Less efficient at scale	Highly efficient at scale

Auto Loader Specifics

- Supports two detection modes: Directory Listing (default) and File Notification (event-driven).

- `pathGlobFilter` : Used to filter input files based on a specific pattern (e.g., `*.png`).
- `cloudFiles.inferSchema = "true"` : Changes default string behavior by sampling the initial batch of files to select precise data types.
- `cloudFiles.schemaEvolutionMode` : Determines behavior for adding new columns (e.g., `addNewColumns`).

Section 8: Spark Structured Streaming

- Captures data streams from growing sources (CDC feeds, message queues, new files landing in cloud storage).
- Utilizes DataStream Reader (`readStream`) and DataStream Writer (`writeStream`).
- **Triggers:**
- *Unspecified (Default):* Executes a micro-batch as soon as the previous one finishes.
- *Processing Time:* Executes at defined intervals (e.g., `trigger(processingTime="2 minutes")`).
- *Available Now:* Processes all available data across multiple micro-batches, then stops (ideal for cost-efficient incremental batch processing).
- **Output Modes:**
- `Append` : Only new rows are incrementally appended to the target table.
- `Complete` : The target table is overwritten entirely with each batch.
- **Checkpointing:**
- Stores stream metadata and tracks processing progress.
- Cannot be shared between separate streams.
- Guarantees **Fault Tolerance** and **Exactly-once** processing when combined with write-ahead logs and idempotent sinks.
- *Note:* Sorting and Deduplication operations are NOT natively supported by standard streaming DataFrames without advanced methods like watermarking.

Section 9: Data Transformation & PySpark Tuning

Data Extraction & Querying

- `CTAS` (Create Table As Select): Automatically infers schema from query results but does *not* support manual schema declaration or specifying additional file options.
- To specify options (e.g., headers, delimiters), use the `USING <data source>` keyword (e.g., `USING CSV OPTIONS(header="true", delimiter=";")`).
- `read_files()`: Simplified file querying function.

SQL Joins & Operators

- **Semi Join:** Checks for existence. Returns left columns only, never duplicates rows.
- **Anti Join:** Returns all records from the left table where no match exists on the right.
- **Null-Safe Equal (`<=>`):** Used to handle comparisons where NULL values are to be treated as equal (e.g., `NULL <=> NULL` is TRUE).

PySpark Optimizations

- **Broadcast Hash Join:** Spark sends a copy of a small table to all worker nodes. This avoids a large shuffle operation and makes the join significantly faster.
- **Partitioning:**
- `coalesce()`: Minimizes partitions by keeping data on the same node (avoids full shuffle).
- `repartition()`: Creates equal partitions across nodes (forces a full shuffle).
- **DataFrames:**
- PySpark DataFrames are immutable.
- `df.withColumnRenamed("old_id", "new_id")`
- `df.fillna(0, subset=["salary"])`
- `df.dropDuplicates()` or `df.distinct()`: Creates a new dataframe containing only unique rows.

Section 10: Troubleshooting & Performance Tuning (Cheat Sheet)

Diagnosing cluster performance issues requires identifying exactly which resource is causing the bottleneck: **Memory**, **Compute**, or **Storage**.

1. Memory (RAM) Bottlenecks

- **The Symptom:** The Spark UI shows "Spill to Disk," or the cluster crashes with an **Out of Memory (OOM)** error.
- **The Cause:** A worker node was handed a partition of data physically larger than its available RAM, forcing it to write the overflow to its local hard drive (which drastically slows down the job).
- **The Fixes:**
- **Increase Shuffle Partitions:** Break the data into smaller, memory-sized chunks by increasing `spark.sql.shuffle.partitions` (default is 200).
- **Fix Data Skew:** Ensure one worker isn't processing a massive disproportionate share of the data (e.g., filtering out `null` keys before joining).
- **Scale Vertically:** Switch the cluster configuration to **Memory-Optimized** nodes.

2. Compute (CPU) Bottlenecks

- **The Symptom:** The job is not spilling to disk, but it is taking a massive amount of time to complete. CPU utilization on the cluster metrics remains pinned at 100%.
- **The Cause:** The mathematical operations are too heavy (e.g., massive cross-joins, complex UDFs, or intensive ML algorithms), and there are not enough CPU cores to process the rows efficiently.
- **The Fixes:**
- **Scale Horizontally:** Add more worker nodes to the cluster to increase the total number of CPU cores.
- **Enable Photon:** Utilize Databricks' C++ engine to drastically speed up CPU-bound SQL operations.
- **Scale Vertically:** Change the cluster to **Compute-Optimized** nodes.

3. Storage (I/O & Disk) Bottlenecks

- **The Symptom:** Tasks spend the vast majority of their execution time in "I/O Wait."
- **The Cause:** The "Small File Problem." Your cloud bucket contains tens of thousands of tiny Parquet files. Spark experiences massive network overhead just opening and closing connections to the storage layer for each tiny file.
- **The Fixes:**
- **Run `OPTIMIZE`:** Compact the tiny files into optimal ~1GB files.
- **Run `ZORDER`:** Reorganize the data on disk so Spark can utilize data skipping to ignore irrelevant files entirely.

- **Cache the Data:** If reading the same storage table multiple times in a single job, use `df.cache()` to pull it into memory once.